

12.03.2024г.

007
Лекция 4

Хоти за контролно - 06.04 от 09:00 - 210/130 химия
26.05 от 15:00 - 210/130 химия
2 часа задачи + 1 час теория

Няма да правим позициониране за `std::cin`, а
за `std::cout` не можем да имаме позициониране.

is.getline (буфер, размер, разделител) // прескага разделителя
is.get (буфер, размер, разделител) // не го прескага
get $\neq$ getline // различни са

Пример: ABCX123
getline (буфер, ..., x) ABC\0 $\searrow$
get (...) ABC\0 $\searrow$

Ако искаме да направим:

getline:
{ get() +
ignore() }

get:

{ getline() +
is.seekg(-1; ios::curr) }

// за домашно да се провери какво ще
стане при eof()

Хвотни файлове

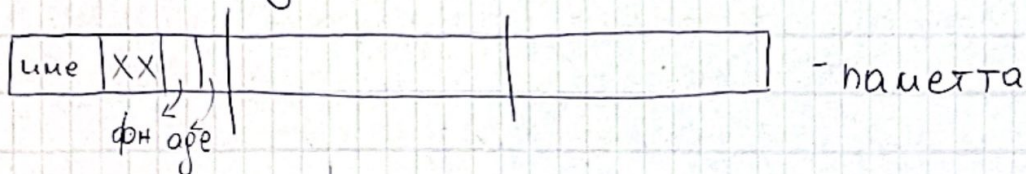
→ с прък ростъи

директно от паметта отива във файл и
обратно памет $\longleftrightarrow$ файл

write (указател към данните, брой байтове)
read(...)

Задача } Имаме масив от студенти
- име с дължина ≤ 10
- ф.н.
- години

Трябва да запазваме във файл.



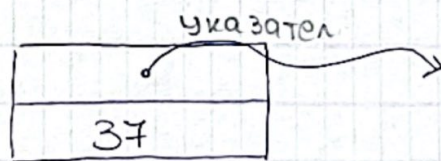
Трябва да четем от файл. // ще връща две неща
- размер на масив
- масив

размер на масива: големината на файл
големината на структурата

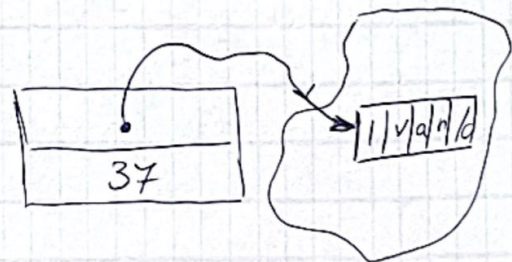
Структури с "външен ресурс"

Имаме човек - име - произволна дължина
- години

```
struct Person
{
    char * name, * nullptr;
    int age;
};
```



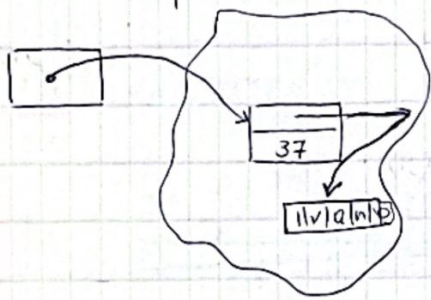
```
Person p;
p.name = new char[5];
strcpy(p.name, "Ivan");
sizeof(P) — не включва
```



stringa, а указателя // това зависи от
операционната система
x64 x32

```
delete[] p.name;
```


Person * p = new Person;

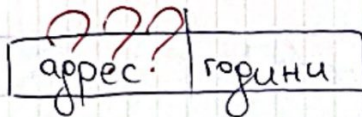


p → name = new char[5];
strcpy(p → name, "Ivan");

delete [] p → name; // това е масив
delete p; // това не е масив,
затова няма []

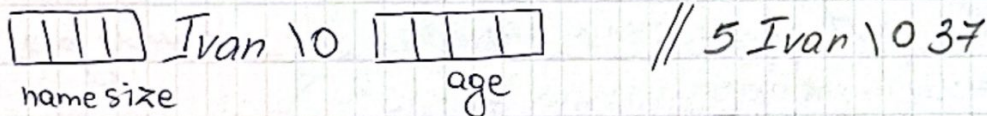
Person P;

write((...) &P, sizeof(P)); // запазваме така указател,
това ще е адрес и години



// така се губят прлзите от
двоичния файл

Запазване на човек

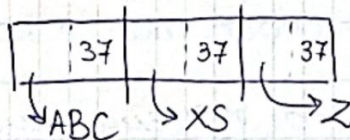


Запазваме дължината на стринга, след това стринга и години

Четене на човек

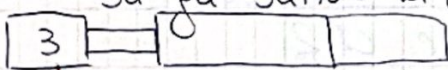
→ Масив от динамични структури

Person p[3]



Всеки Person е с
различна големина.

за да запазим в файл:



брой
елементи



ios::binary

(In)

(It)

(In) ^{2 байта}
get() → x | y байтове

ios::binary
x
y

Не тече новия рег, тече x байт и y байт отделно.

ios::ate

std::ofstream ("ABC", std::ios::ate)

// позиционира на началото
на файла и останалото trunk()

затова:

std::ofstream ("ABC", std::ios::in | std::ios::ate);

// поставя път указателя накрая и монитора
се рвиним напред

Работа с fstream

fstream ... ios::in

≈ ifstream

→ позволява писане и четене едновременно
// но това носи опасност, ако е голям файл

Зарата / функция, която приема име на файл и ch1 и
ch2 (символи). Искане във файла, всяко
срещане на ch1 да се замести с ch2.

// трябва едновременно да четем и да
пишем

`fstream.... (...., ios::in / ios::out)`



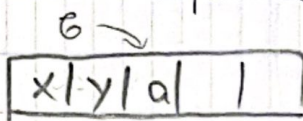
указателят е един * // приемоме, че в общия случай е така

`tellp() == tellr()`



При `fstream`

при входна операция, след изходна трябва да синхронизирате буфера.



get ук // трябва да се върнем и позиция назад

Трябва да извикаме `flush()`; `seekg(tellr())` също може да се използва, защото става `flush()`

Злен - функции

Имаме функции в тялото на структура.

Пример: `struct Point {`

`int x;`

`int y;`

`bool isInFirstQ... () { return x ≥ 0 & y ≥ 0 }`

`}`

`Point p { 7, 9 }`

`p.isInFirstQ... ()`

`Point * p = new Point ...`

`p → isInFirst...`

struct

struct A {
void f() { }
};

преобразува
се като
външна
функция

; добавя се още един параметър,
който е обекта, който се

struct A {
};

const // не можем да извиква
променяме указателя

f(A* this) // приема указател, който
винаги е this

struct A {
int x = 7;
void print() {
cout << x;
};

const

Print (A* this)
cout << this->x;

this е запазена дума
// член-функция, която ще отпечата адреса // this->x

⚠ член-функции

- Извикват се от обект (трябва да има обект)
- Работят директно с член-данните (променливите, които са в обекта)

→ Компиляторът ги преобразува във външна функция.


```

struct A {
    int x=10;
    void print() const
    { cout << x; }
};

```

```

test F(const A& ref)
    ref.print();

```

// няма да може да се компилира
защото не знаем в какъв клас-
функцията дали няма да променим
A ref.

Зато ва :

Константна клас функция

```

A {
    void f() const { }
};

```

• Не променя клас-ранните.

• Може да се извиква от константни обекти/
референции/
указатели

• Константните клас-функции ^{в структура} могат да извикват само
~~константни~~ клас-функции на структурата, които са
константни.

```

struct A {
    f();
    g() const;
    h() const;
};

```



```

struct A {
    f() { ... }
    f() const { ... }
};
A obj;
obj.f();

const A obj1;
obj1.f();
    
```

извиква

// зависи как поробаме A (кого const или не)

понение е const, се транслира

```

void f(const A* const this)
    
```

↓

дам функцията е константна

↓

указателят не се мести

! this++ не може;

```

{ Point p{3,4}
    
```

заделя се памет

задават се стойности на променливите

конструктор

можем да го правим в две различни функции

излизват се външните ресурси

→ освобождава се паметта за обекта

```

struct A {
    int a;
    int b;
    A(int x) { a = b = x; }
    A(int aParam, int bParam) { a = aParam; b = bParam; }
};
    
```

← конструктори

A obj(3) → заделя памет
 → извиква конструктор I

// можем да имаме повече от един конструктор

A obj(3, 9) // ще се извика II при конструктор

Важно за конструктор:

- извиква се при създаване на обекта;
- има същото име като класа, структурата
- няма тип на връщане
- можем да имаме много конструктори
- конструктор, който е без параметри се нарича default конструктор.

```
struct A {
```

```
int a, b;
```

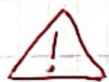
```
  A(int)
```

```
  A(int, int)
```

A() — по подразбиране е този конструктор

```
A arr[5];
```

→ на този обект му се извиква пет пъти default constructor



Ако в структурата не е дефиниран нито един конструктор, то компилаторът създава default-ен конструктор.

```
struct A {
```

```
int x;
```

```
}
```

```
struct A {
```

```
int x;
```

компиляторът
създава A() {};

```
}
```

! Извикването на default-ен конструктор.

Test t3;

~~Test t3();~~

→ това е функция, която връща Test

→ това извиква default-ен конструктор

Защо няма скоби? — Защото със скоби е деклариране на функция

за теория

Ако имаме масив от структури, то оставя default-конструктор. Ако нямаме default-конструктор, а някакви други, няма да се компилира, защото трябва да е default.

Деструктор

- отговаря за затварянето на всякакви външни ресурси.
- можем да имаме точно 1.
- същото име като структурата, но има в началото символът $\sim$ (тилда)

`{ Point p(3,4) → зорече памет`
`→ Point(3:int, int) // извиква конструктор`

③ ~ Point() // инициализация деструктор
→ освобождение

Наслеб от структури:

```

Point p[3];
Point() Point() Point() // toba ye ee
                           onera q

```

} ~Point(), ~Point(), ~Point() // това ще се
отнеже

за теория

Имате отпечетване в дефинир. конструктор и деструктор на структурата A.
Какво ще отпечата корът?

```
{ Point * p = new Point (); // може вместо () ; само ;  
                                Point ()  
    delete p; ~Point ()  
}
```

```
Point * arr = new Point [3] // ще отпечата 3x Point ()  
delete [] arr; // ще отпечата 3x ~Point ()
```

new — зарежа памет
— извиква ~~де~~ конструктор

delete — деструктор се извиква
— освобождава се памет

при конфликт на
имена можем да
използваме *this* →